

ReFit: Repairing Speculative Drafters for Post-Trained Language Models

Anonymous ACL submission

Abstract

Speculative decoding with target-conditioned drafters such as EAGLE-3 is among the fastest ways to serve large language models, and the drafter released with a base model is routinely reused for its many post-trained derivatives. We show that this reuse breaks for an important and growing class of models: derivatives distilled from a stronger reasoning teacher. In a census of 174 public derivatives, quantized, merged and RL-tuned models retain 98–100% of the family drafter’s acceptance, while DeepSeek-R1-Distill-Llama-8B and Llama-3.1-Nemotron-Nano-8B lose up to 30%, erasing most of the speedup. Post-training changes both the hidden states the drafter reads and the tokens it must predict, and the damage concentrates in the drafter’s *feature interface*, the projection through which it reads the target. We introduce *ReFit*, which re-fits a family drafter on the derivative’s own responses to public instructions, without any of the original training data. Re-fitting the interface alone (12% of the drafter’s parameters) closes 60% of the gap to a dedicated drafter trained specifically for the derivative, and a full warm start closes 72%, using 11 GPU-hours, over 400× less compute than the public recipe for a dedicated drafter. On SPEED-Bench and MATH-500, ReFit raises the end-to-end speedup of R1-Distill from $1.31\times$ to $1.83\times$ (dedicated drafter: $2.05\times$) and of Nemotron-Nano from $1.29\times$ to $1.80\times$, outperforming independent draft models.

1 Introduction

Speculative decoding has become the standard way to serve large language models (LLMs) quickly: a small drafter proposes several tokens, and the target model verifies all of them in a single forward pass, accelerating generation without changing its output (Leviathan et al., 2023; Chen et al., 2023). The fastest drafters today are *target-conditioned*. Rather than modeling text from scratch, they read

the target’s own hidden states and predict its next tokens from them (Cai et al., 2024; Li et al., 2024b, 2025; Chen et al., 2026b). EAGLE-3, for instance, fuses features from the target’s lower, middle and upper layers and reports speedups of up to $6.5\times$ (Li et al., 2025). This tight coupling is also their price: each drafter is trained for one specific target, on hundreds of thousands of examples, for tens of epochs.

Open-weight models, however, rarely stay single models. A model family quickly acquires instruction-tuned releases, quantized and merged variants, preference-optimized and RL-tuned checkpoints, and reasoning distillations; a popular base model such as Llama-3.1-8B spawns thousands of such derivatives on public model hubs (Horwitz et al., 2025). Training a dedicated drafter for each one is out of reach, so serving stacks reuse the drafter released for the family’s instruction model, the *family drafter*, for all of its relatives (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). Practitioners make this choice every day, yet two questions behind it remain open: when does a family drafter keep working for a derivative, and what can be done when it does not?

We answer the first question with a census of 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Figure 1a). For most of the ecosystem, reuse is reliable. Quantized, merged, ablated and on-policy RL derivatives retain 98–100% of the family drafter’s acceptance, in line with evidence that on-policy training keeps models close to their initialization (Shenfeld et al., 2026; Chen et al., 2026a). Reuse fails, however, for precisely the derivatives that are most expensive to serve: models distilled from stronger reasoning teachers. DeepSeek-R1-Distill-Llama-8B and Llama-3.1-Nemotron-Nano-8B lose 26–30% of first-token acceptance under both EAGLE-3 and DFlash drafters. For R1-Distill, the speedup collapses from $2.05\times$ with a dedicated drafter to

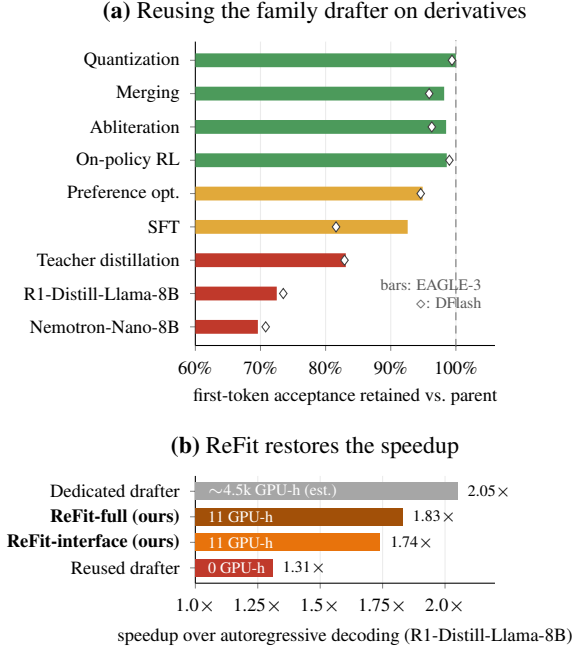


Figure 1: (a) First-token acceptance of the Llama-3.1-8B-Instruct family drafters on public derivatives, relative to the parent model (174 derivatives grouped by post-training history; EAGLE-3 bars, DFlash diamonds). Weight-space edits and on-policy RL leave the drafter intact; distillation from a reasoning teacher breaks it. (b) On R1-Distill-Llama-8B, ReFit recovers most of the speedup of a dedicated drafter at a small fraction of its training cost.

1.31 $\times$ (Figure 1b). Reasoning models routinely emit thousands of tokens per query, so this lost speedup costs the most exactly where acceleration matters most.

We then show where the damage lies. Post-training changes two things the drafter depends on: the target’s hidden states, which the drafter reads, and the target’s next-token policy, which it must predict. A controlled crossover shows that both shifts are substantial. Yet the drafter accesses the target through a single learned map, the *feature interface*, a dense projection that compresses the target’s multi-layer hidden states into the drafter’s input space. Re-fitting this interface alone recovers most of what re-training the entire drafter achieves, while the drafter’s decoder layer and language-modeling head transfer largely intact. This observation leads to *ReFit* (Figure 3): elicit the derivative’s own responses to public instructions, then re-fit the family drafter on them with its native training objective, updating either the interface alone (ReFit-interface) or the whole warm-started drafter (ReFit-

full). ReFit needs no access to the derivative’s or the drafter’s training data, no new architecture, and no change to the serving engine.

ReFit is highly effective. On R1-Distill-Llama-8B, ReFit-interface closes 60% of the gap between the reused family drafter and a dedicated drafter trained by the EAGLE authors, and ReFit-full closes 72%. With 11 GPU-hours of compute, over $400\times$ less than the public recipe for a dedicated drafter, ReFit lifts the end-to-end speedup from $1.31\times$ to $1.83\times$. On Nemotron-Nano, for which no dedicated drafter exists, it lifts the speedup from $1.29\times$ to $1.80\times$. Repaired drafters are faster than independent small draft models and training-free drafting, and the gains hold across drafter checkpoints and drafter families. In summary, we make three contributions:

- We present a census of family-drafter reuse across 174 public derivatives and identify off-policy distillation as the post-training regime that breaks target-conditioned drafters (§3).
- We trace the failure to shifted target features and policy, show that it concentrates in the drafter’s feature interface, and propose ReFit, a simple recipe that repairs the family drafter from the derivative’s own outputs without any original training data (§3–§4).
- We show on SPEED-Bench and MATH-500 that ReFit recovers most of a dedicated drafter’s speedup at a small fraction of its cost, across targets, drafter checkpoints and drafter families (§6).

2 Background: Target-Conditioned Drafters

Speculative decoding. Given a prefix $y_{\leq t}$, a drafter proposes K tokens $\hat{y}_{t+1:t+K}$. The target scores all of them in one forward pass and accepts the longest prefix that agrees with its own predictions, plus one bonus token that it generates itself (Leviathan et al., 2023; Chen et al., 2023). The output is identical to decoding with the target alone. Efficiency is governed by the acceptance length τ , the average number of tokens produced per target forward pass, and by the cost of drafting. With per-step costs c_T for the target and c_D for the drafter, the speedup is approximately $\tau c_T / (c_T + K c_D)$. A good drafter is therefore both *accurate* (high τ) and *cheap* (low c_D). We also report first-token acceptance p_1 , the rate at which the target accepts the

first drafted token; unlike τ , it does not depend on how long the target’s responses are.

Target-conditioned drafting. Target-conditioned drafters achieve both properties by reusing the target’s computation. Medusa attaches extra decoding heads to the target’s final layer (Cai et al., 2024), and EAGLE drafts autoregressively at the feature level (Li et al., 2024b,a). EAGLE-3 (Li et al., 2025) reads hidden states $h_t^{\text{low}}, h_t^{\text{mid}}, h_t^{\text{high}} \in \mathbb{R}^d$ from three depths of the target and maps them through a learned *feature interface* $W_{\text{fc}} \in \mathbb{R}^{d \times 3d}$:

$$g_t = W_{\text{fc}} [h_t^{\text{low}}; h_t^{\text{mid}}; h_t^{\text{high}}]. \quad (1)$$

The fused feature g_t is combined with the embedding of the next token and passed through a single decoder layer and a language-modeling head over a reduced draft vocabulary, producing the draft distribution q_θ . EAGLE-3 is trained with *training-time test*: the drafter is unrolled on its own predictions for $k = 1, \dots, k_{\text{max}}$ steps and matched to the target’s next-token distribution p_T at each step,

$$\mathcal{L}(\theta) = \sum_{k=1}^{k_{\text{max}}} \mathbb{E}_t \text{KL} \left(p_T(\cdot \mid y_{\leq t+k}) \parallel q_\theta^{(k)} \right), \quad (2)$$

so that it learns to draft from its own predictions as it does at inference. DFlash (Chen et al., 2026b) drafts an entire block in parallel with a lightweight diffusion model, but it reads the target through the same kind of multi-layer feature projection.

Family drafters. A drafter is trained once for a family’s instruction model and released publicly. Serving engines then load it for any model that shares that model’s architecture and tokenizer (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). We call the instruction model the *parent* T , its drafter the *family drafter* D_{θ_0} , and the model being served the *derivative* T' .

3 When Family Drafters Fail

A census of derivatives. We collect 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Grattafiori et al., 2024; Yang et al., 2025) from the Hugging Face hub, all of which load with the architecture and tokenizer of the instruction-tuned parent and its released drafters. We label each by its post-training history as described in its model card: quantization, merging, ablation,

supervised fine-tuning (SFT), preference optimization, on-policy reinforcement learning, or distillation from a teacher. For every derivative, we serve the family EAGLE-3 and DFlash drafters with vLLM (Kwon et al., 2023) and measure first-token acceptance on identical prompts for the derivative and its parent. We report retention, the derivative’s acceptance divided by the parent’s.

Distillation breaks the drafter. Figure 1a shows a clear ordering. Edits in weight space leave the drafter intact: quantized derivatives retain 100% of the parent’s acceptance, merged ones 98% and ablated ones 99%. On-policy RL does not disturb it either (99%). Off-policy supervision erodes acceptance progressively, from 95% for preference optimization and 93% for SFT to 83% for distillation from a teacher. The most severe losses occur for the two widely used reasoning distills. R1-Distill-Llama-8B, trained on the outputs of DeepSeek-R1 (DeepSeek-AI, 2025), retains 72.5% under EAGLE-3 and 73.5% under DFlash. Nemotron-Nano-8B (Bercovich et al., 2025) retains 69.6% and 70.8%. This ordering mirrors how far each regime moves a model from its initialization. On-policy RL learns from the model’s own samples and stays close to it (Shenfeld et al., 2026; Chen et al., 2026a), whereas distillation fits the model to a different model’s distribution (Hinton et al., 2015; Kim and Rush, 2016) and reshapes its internal representations.

The cost of this loss is substantial. On R1-Distill, the reused drafter reaches an acceptance length of only 1.76 tokens per step on SPEED-Bench, compared with 2.85 for a dedicated EAGLE-3 drafter that the EAGLE authors trained on R1-Distill. In end-to-end decoding, this is the difference between $1.31 \times$ and $2.05 \times$ (§6). A dedicated drafter is rarely an option: the public EAGLE-3 recipe trains on more than half a million examples for tens of epochs, and for most derivatives, including Nemotron-Nano, no dedicated drafter exists at all. Practitioners can, however, tell cheaply when reuse is at risk: acceptance measured on just 16 requests ranks the degraded derivatives in our populations with an AUROC between 0.91 and 1.00.

Both the features and the policy move. A drafter can fail on a derivative for two reasons: the derivative presents it with different hidden states, or the derivative wants different next tokens. We separate the two with a fixed-text crossover. On 64 contexts generated by each derivative, we feed the

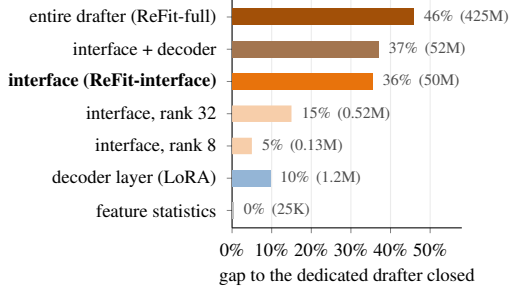


Figure 2: **Where to repair.** Fraction of the acceptance-length gap between the reused family drafter and a dedicated drafter closed by re-fitting each component (R1-Distill-Llama-8B, SPEED-Bench, 256 repair examples; trainable parameters in parentheses). The dense feature interface carries most of the recoverable gain.

family drafter either the parent’s or the derivative’s hidden states and score its first prediction against either model’s next token. Swapping in the derivative’s hidden states alone lowers agreement by 2.0 points on R1-Distill (95% CI $[-2.5, -1.4]$) and by 6.9 points on Nemotron-Nano ($[-7.7, -6.2]$). Swapping in the derivative’s next-token policy lowers it by another 3.0 ($[-3.5, -2.6]$) and 3.7 points ($[-4.3, -3.1]$). The family drafter thus misreads the derivative’s features even when the desired token is unchanged, and it also aims at the wrong tokens. An effective repair must teach the drafter to read the derivative’s features, under supervision from the derivative itself.

The damage concentrates in the interface. Which part of the drafter should learn? We re-fit individual components of the R1-Distill family drafter on 256 of the derivative’s responses (§4) and measure how much of the gap to the dedicated drafter each closes (Figure 2). Adapting the decoder layer with LoRA (Hu et al., 2022) closes only 10% of the gap, and recalibrating the statistics of the target’s features closes none. Re-fitting the dense feature interface closes 36%, nearly as much as updating the interface and decoder together (37%) and most of what re-training the entire drafter achieves (46%). Low-rank updates of the interface fall well short (5–15%). Post-training therefore does not merely perturb the derivative’s features; it requires a full re-mapping of how they should be read. The drafter’s decoder and head remain good at drafting. What fails is the interface, which no longer understands the model it serves.

4 ReFit: Repairing the Family Drafter

Given a derivative $\mathcal{T}'$ and the family drafter D_{θ_0} of its parent, ReFit produces a drafter D_{θ} matched to $\mathcal{T}'$ without the data used to train either model. The recipe has three steps (Figure 3).

Eliciting the derivative. The drafter must predict what $\mathcal{T}'$ will generate, so we let $\mathcal{T}'$ show it. We sample instructions x from a public instruction corpus. We use 16k instructions from Alpaca (Taori et al., 2023), deduplicated against all evaluation prompts. We render each instruction with the derivative’s chat template and greedily decode a response $y' \sim \mathcal{T}'(\cdot | x)$ of up to 512 tokens. For a reasoning distill, these responses are long reasoning traces, exactly the text the drafter will face in deployment. This mirrors sequence-level distillation (Kim and Rush, 2016) and drafter alignment (Zhou et al., 2024; Goel et al., 2024), with one difference: only the drafter learns, and only to follow a fixed target.

Re-fitting the drafter. For each pair (x, y') , a single teacher-forced pass of $\mathcal{T}'$ provides, at every response position, the three hidden states that enter Eq. (1) and the next-token distribution $p_{\mathcal{T}'}$. These two signals address exactly the two shifts identified in §3: the drafter learns to read $\mathcal{T}'$ ’s features and to predict $\mathcal{T}'$ ’s tokens. We minimize the drafter’s native training-time-test objective (Eq. 2) with $p_{\mathcal{T}'}$ in place of p_T and $k_{\max} = 3$, starting from the family drafter’s parameters θ_0 . We consider two operating points:

- **ReFit-interface** updates only W_{fc} , 50M parameters or 12% of the drafter, and keeps the decoder layer and head frozen.
- **ReFit-full** warm-starts all drafter parameters from θ_0 ; the token embedding remains shared with $\mathcal{T}'$.

Both variants inherit everything the family drafter already knows about drafting and only teach it to read the new target. This warm start is essential. The same architecture trained from scratch on the same data and budget falls below even the reused drafter (§6).

Cost and deployment. ReFit takes one pass over 16k examples, or 7.9M response tokens. Generating the responses and training together take about 11 A40 GPU-hours. The repaired drafter has the same architecture, vocabulary and file format as

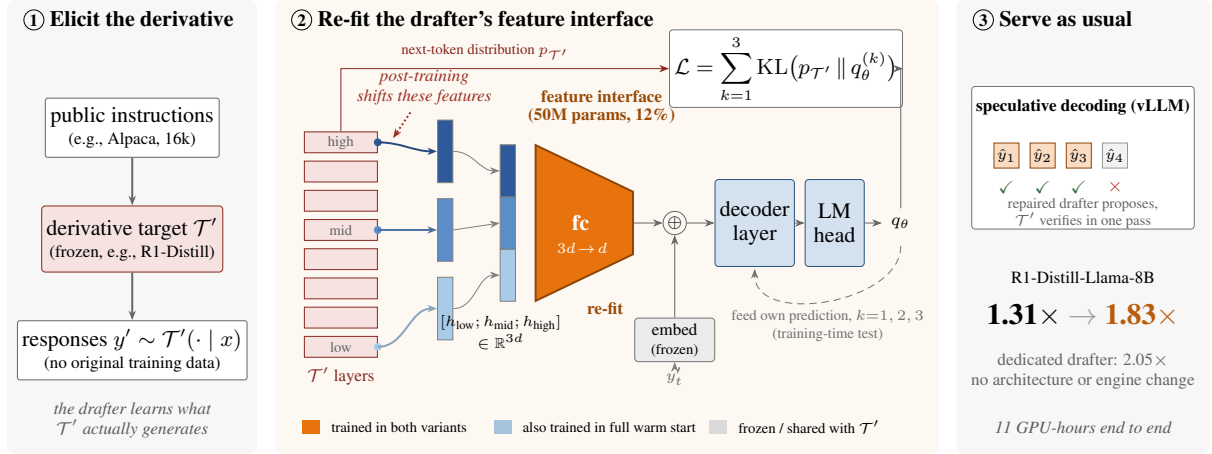


Figure 3: **ReFit**. ① The derivative target $\mathcal{T}'$ answers public instructions, producing data that reflect what it actually generates; none of its training data, nor the drafter’s, is needed. ② A teacher-forced pass of $\mathcal{T}'$ exposes the multi-layer hidden states that the drafter reads and the next-token distribution it must match. Post-training has shifted both, so we re-fit the drafter’s feature interface (orange, ReFit-interface) with its native training-time-test objective, optionally warm-starting the decoder layer and head as well (blue, ReFit-full). ③ The repaired drafter keeps its architecture and format, so it is served with an unchanged engine.

the family drafter, so it is a drop-in replacement in the serving engine.

5 Experimental Setup

Targets and drafters. We study the two reasoning distills from the census: DeepSeek-R1-Distill-Llama-8B, distilled from DeepSeek-R1 into Llama-3.1-8B (DeepSeek-AI, 2025), and Llama-3.1-Nemotron-Nano-8B-v1, derived from Llama-3.1-8B-Instruct through distillation and reasoning post-training (Bercovich et al., 2025). Both share the Llama-3.1 architecture and tokenizer, and hence the family drafters of Llama-3.1-8B-Instruct. Our main family drafter is the EAGLE-3 drafter that the EAGLE authors released for Llama-3.1-8B-Instruct (Li et al., 2025). We also repair the production EAGLE-3 drafter distributed with vLLM’s speculators library (Red Hat AI and vLLM Project, 2025) and the DFlash drafter (Chen et al., 2026b). As an upper reference, we use the dedicated EAGLE-3 drafter that the EAGLE authors trained specifically for R1-Distill-Llama-8B.

Baselines. We compare against (i) reusing the family drafter unchanged; (ii) an independent draft model, Llama-3.2-1B-Instruct (Grattafiori et al., 2024), which shares the targets’ tokenizer; (iii) n-gram prompt lookup (Saxena, 2023), a training-free drafter built into vLLM; and (iv) training the same EAGLE-3 architecture from scratch on our repair data with the same budget. All speculative methods propose $K=4$ tokens per step.

Benchmarks and metrics. We evaluate on SPEED-Bench (Abramovich et al., 2026), a unified benchmark for speculative decoding, using a stratified 128-prompt sample of its qualitative split, which spans eleven domains from coding and math to writing, roleplay and multilingual chat. For long-form reasoning, we use MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024). Prompts are rendered with each target’s chat template, and responses are generated greedily for up to 512 tokens. We report p_1 , τ (including the bonus token) and, for R1-Distill, the *gap closed*, $(\tau_{\text{repair}} - \tau_{\text{reuse}}) / (\tau_{\text{dedicated}} - \tau_{\text{reuse}})$. We measure end-to-end wall-clock speedup over autoregressive decoding in vLLM on NVIDIA A40 GPUs at batch sizes 1 and 8. R1-Distill results on SPEED-Bench average three training seeds; confidence intervals are 95% paired bootstrap intervals over prompts (and seeds). Further details are in Appendix A.

6 Results

6.1 Main Results

Table 1 summarizes our results. On R1-Distill-Llama-8B, ReFit-interface raises the acceptance length on SPEED-Bench from 1.76 to 2.41 (paired gain 0.650 [0.607, 0.692]) and first-token acceptance from 43.0% to 62.6%, closing 59.9% [57.6, 62.4] of the gap to the dedicated drafter while keeping the decoder fixed. ReFit-full raises the acceptance length to 2.54 (0.778 [0.731, 0.823]) and first-token acceptance

Method	Train GPU-h	R1-Distill-Llama-8B					Nemotron-Nano-8B				
		p_1 (%)	τ		Speedup		p_1 (%)	τ		Speedup	
			SPEED	MATH	bs 1	bs 8		SPEED	MATH	bs 1	bs 8
Autoregressive	–	–	1.00	1.00	1.00×	1.00×	–	1.00	1.00	1.00×	1.00×
Independent drafter (1B)	–	61.3	2.44	2.97	1.26×	1.11×	64.7	2.65	2.84	1.37×	1.24×
Family drafter, reused	0	43.0	1.76	1.93	1.31×	1.07×	42.9	1.76	1.70	1.29×	1.13×
ReFit-interface (ours)	11	62.6	2.41	2.77	1.74×	1.27×	61.4	2.42	2.72	1.69×	1.39×
ReFit-full (ours)	11	65.0	2.54	2.97	1.83×	1.29×	62.9	2.49	2.91	1.80×	1.42×
Dedicated drafter (reference)	~4.5k	70.5	2.85	3.84	2.05×	1.41×	<i>not available</i>				

Table 1: **Main results.** First-token acceptance p_1 , acceptance length τ and end-to-end speedup over autoregressive decoding (vLLM, A40, batch sizes 1 and 8). MATH is MATH-500 for R1-Distill and a 64-problem MATH sample for Nemotron-Nano. ReFit improves every acceptance metric over reuse with 95% confidence intervals excluding zero, and ReFit-full is the fastest method that does not require a dedicated drafter. Training cost includes generating the repair data; the dedicated drafter’s cost is estimated for the public EAGLE-3 recipe.

to 65.0%, within 5.5 points of the dedicated drafter, closing 71.7% [69.2, 74.3] of the gap. The gains are equally strong on long mathematical reasoning: on MATH-500, ReFit raises τ from 1.93 to 2.77 and 2.97, closing 44% and 54% of the gap. On Nemotron-Nano, where no dedicated drafter exists and repair is the only way to obtain a matched drafter, ReFit-interface and ReFit-full raise τ by 0.655 [0.604, 0.707] and 0.724 [0.662, 0.787] on SPEED-Bench, and by 1.01 and 1.21 on MATH.

These gains carry over to wall-clock time. At batch size 1, the repaired drafters accelerate R1-Distill by 1.83×

The family drafter is the asset. Training the same EAGLE-3 architecture from scratch on the repair data reaches an acceptance length of only 1.52 on SPEED-Bench, below even the unrepaired family drafter and a full token per step below ReFit-full. A drafter’s ability to draft is expensive to acquire and is exactly what the family drafter already has. ReFit preserves this ability and spends its budget on teaching the drafter to read its new target.

ReFit versus other drafters. Training-free n-gram lookup rarely finds long matches in open-ended generation and stays below 1.6 tokens per

step on both targets. The independent Llama-3.2-1B drafter predicts the derivatives well, since it is itself a capable language model, but it must run sixteen transformer layers for every drafted token. Its drafting cost consumes most of its acceptance, and it reaches only 1.26×

6.2 Scaling and Cost

Figure 4a shows that ReFit improves steadily with data. With the production drafter, the gap closed by ReFit-full grows from 46% with 0.5k examples to 58% with 4k, 66% with 16k and 74% with 64k. Every increase in data helps, and the curve is still rising at 64k examples. The EAGLE authors’ family drafter starts from a better position and stays ahead at every budget (64% at 4k, 72% at 16k). ReFit-interface follows a parallel curve 12–15 points lower and thus captures most of the benefit of the full warm start with an eighth of the trainable parameters.

Figure 4b places these results in terms of compute. Even 0.45 GPU-hours of repair on 256 examples already yield a 1.67×

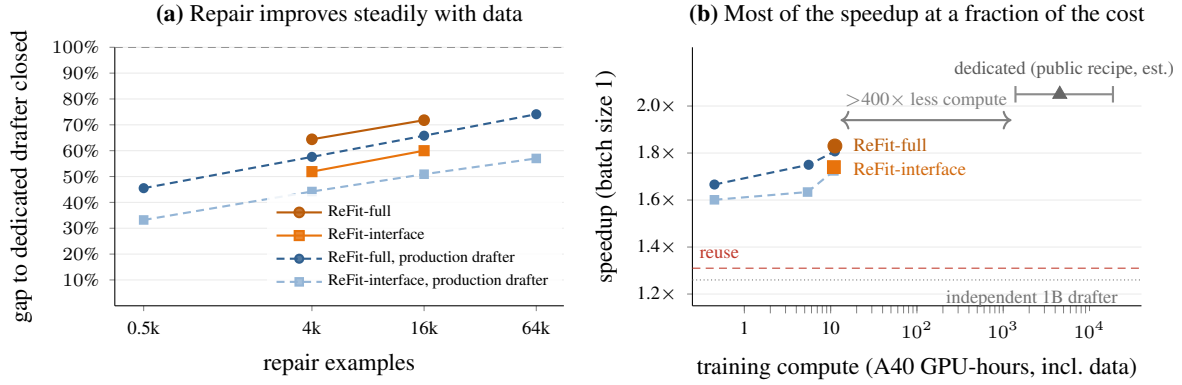


Figure 4: **(a)** Gap to the dedicated drafter closed on R1-Distill-Llama-8B as a function of the number of repair examples, for the EAGLE authors’ family drafter (solid) and the production drafter (dashed). **(b)** Batch-size-1 speedup against training compute, including data generation. ReFit attains most of the dedicated drafter’s speedup with orders of magnitude less compute; the dedicated drafter’s range spans 10 to 40 epochs of the public recipe.

Family drafter	Target	Reuse	ReFit	
			interface	full
EAGLE-3 (authors)	R1	1.76	2.41	2.54
	Nemotron	1.76	2.42	2.49
EAGLE-3 (production)	R1	1.73	2.30	2.47
	Nemotron	1.82	2.32	2.42
DFlash (256 ex.)	R1	2.08	2.22	2.31
	Nemotron	2.08	2.24	2.36

Table 2: ReFit generalizes across drafter checkpoints and drafter families. Acceptance length on SPEED-Bench; EAGLE-3 drafters use 16k repair examples, and DFlash drafts blocks of ten tokens and is repaired with only 256 examples.

6.3 Generality

Table 2 shows that the benefits of ReFit do not depend on a particular drafter. Repairing the production EAGLE-3 drafter distributed with vLLM yields the same picture as repairing the drafter of the EAGLE authors. ReFit-full closes 66% of the R1-Distill gap, ReFit-interface closes 51%, and the end-to-end speedups are $1.81\times$ and $1.72\times$. The two family drafters break equally under reuse, but the EAGLE authors’ drafter gains more from repair, by 0.081 [0.048, 0.115] tokens per step for ReFit-interface and 0.043 [0.014, 0.073] for ReFit-full. The method also transfers to a different drafter family. DFlash replaces autoregressive drafting with parallel block diffusion but reads the target through the same kind of feature interface, and repairing it with only 256 examples raises its acceptance length by 11–13% on both targets. Across all six settings, the pattern of Figure 2 holds: the interface carries most of the repair, and a full warm start adds the rest.

7 Related Work

Speculative decoding. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) has been extended with tree-structured verification (Miao et al., 2024; Chen et al., 2024), self-speculation (Zhang et al., 2024a), and model-free drafting from n-grams, retrieval or suffix structures (Fu et al., 2024; He et al., 2024; Saxena, 2023; Oliaro et al., 2024); see Xia et al. (2024) for a survey. The strongest drafters condition on the target’s hidden states, from Medusa and Hydra heads (Cai et al., 2024; Ankner et al., 2024) to the EAGLE family (Li et al., 2024b,a, 2025; EAGLE Team et al., 2026), harmonized representations (Zhang et al., 2024b), parallel drafters (Jiang et al., 2026) and block-diffusion drafters (Chen et al., 2026b). Open frameworks such as SpecForge (Li et al., 2026a) and speculators (Red Hat AI and vLLM Project, 2025) make training such drafters possible but not cheap, and complementary work reduces their cost by pruning the draft vocabulary (Zhao et al., 2025; Goel et al., 2025). We take these drafters as given and ask how to keep them working when their target changes.

Adapting drafters. DistillSpec (Zhou et al., 2024) distills a draft model from its target, online speculative decoding (Liu et al., 2024) keeps updating it on served traffic, and Draft-OPD (Lei et al., 2026) trains drafters on their own on-policy proposals. Other work aligns draft models with chat-tuned targets (Goel et al., 2024), trains drafters for new domains (Hong et al., 2025), adapts them with parameter- and data-efficient updates (Lin et al., 2026), shares frozen drafts across evolving targets

(Li et al., 2026b), or evolves them online across vocabularies and targets (Ramakrishnan et al., 2025; Zhang et al., 2026). These approaches treat the drafter as a model to be re-aligned as a whole, or change its architecture or training regime. For target-conditioned drafters, we show that the misalignment caused by post-training concentrates in a single existing component, the feature interface. Re-fitting it offline from public instructions recovers most of a dedicated drafter’s speedup, without traffic logs, new modules or original training data.

Post-training and representation drift. Fine-tuning distorts pretrained features (Kumar et al., 2022), and the extent depends on the training signal: on-policy RL forgets less than supervised training on fixed data (Shenfeld et al., 2026; Chen et al., 2026a). The open-model ecosystem consists largely of such derivatives (Horwitz et al., 2025), and reasoning distills (DeepSeek-AI, 2025; Bercovich et al., 2025) and multi-stage post-training pipelines (Lambert et al., 2024) are among its most widely deployed members. Our census connects these findings to inference efficiency. The same regimes that move a model furthest from its initialization are the ones that break the drafters built for it.

8 Conclusion

Family drafters make speculative decoding practical across the open-model ecosystem, but they silently lose much of their value on derivatives distilled from stronger reasoning teachers. We showed that post-training shifts both the features a drafter reads and the tokens it must predict, that the resulting damage concentrates in the drafter’s feature interface, and that re-fitting this interface on the derivative’s own responses to public instructions restores most of a dedicated drafter’s speedup for a few GPU-hours. ReFit turns per-derivative drafting from an expensive training project into a routine step that can accompany every model release. More broadly, our results suggest that the coupling that makes target-conditioned drafters fast need not make them brittle: when the target moves, it suffices to teach the drafter how to read it again.

Limitations

Our study focuses on 8B-scale derivatives of the Llama-3.1 and Qwen3 families and on the EAGLE-3 and DFlash drafters, which are among the most widely deployed configurations. ReFit requires

running the derivative to generate responses; this cost is small and is included in all our compute figures. We measure wall-clock speedups with greedy decoding on NVIDIA A40 GPUs in vLLM; absolute speedups vary with hardware, batch size and decoding configuration.

Ethical Considerations

ReFit leaves the target model’s weights and outputs unchanged: speculative decoding is lossless, so repair affects only serving cost, not what the model generates. By reducing the compute needed to serve reasoning models, it lowers the energy cost of deploying them. All models and prompt sources are public and used under their licenses, and no private user data is involved.

References

- Talor Abramovich, Maor Ashkenazi, Izzy Putterman, Benjamin Chislett, Tiyasa Mitra, Bitu Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. 2026. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *International Conference on Machine Learning (ICML)*.
- Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*.
- Akhiad Bercovich et al. 2025. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. 2026a. Retaining by doing: The role of on-policy data in mitigating forgetting. In *International Conference on Machine Learning (ICML)*.
- Jian Chen, Yesheng Liang, and Zhijian Liu. 2026b. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*.
- Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, robust, and

616	hardware-aware speculative decoding. <i>arXiv preprint</i>	Weifan Jiang, Krishna Teja Chitty-Venkata, Megan	667
617	<i>arXiv:2402.12374</i> .	Flynn, Reed Meyerson, Zhenting Qi, Tianyu Wu,	668
618	DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing rea-	Eldar Kurtic, Minlan Yu, and Alexandre Marques.	669
619	soning capability in LLMs via reinforcement learning.	2026. H-Spec: Parallel speculative decoding	670
620	<i>arXiv preprint arXiv:2501.12948</i> .	without a drafter-side KV cache. <i>arXiv preprint</i>	671
621	EAGLE Team, vLLM Team, and TorchSpec Team.	<i>arXiv:2609.24197</i> .	672
622	2026. EAGLE 3.1: Advancing speculative decod-	Yoon Kim and Alexander M. Rush. 2016. Sequence-	673
623	ing. https://lightseek.org/blog/eagle-3-1.	level knowledge distillation. In <i>Proceedings of the</i>	674
624	html .	<i>Conference on Empirical Methods in Natural Lan-</i>	675
625	Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang.	<i>guage Processing (EMNLP)</i> .	676
626	2024. Break the sequential dependency of LLM	Ananya Kumar, Aditi Raghunathan, Robbie Jones,	677
627	inference using lookahead decoding. In <i>International</i>	Tengyu Ma, and Percy Liang. 2022. Fine-tuning can	678
628	<i>Conference on Machine Learning (ICML)</i> .	distort pretrained features and underperform out-of-	679
629	Raghavv Goel, Mukul Gagrani, Wonseok Jeon, Juny-	distribution. In <i>International Conference on Learn-</i>	680
630	young Park, Mingu Lee, and Christopher Lott. 2024.	<i>ing Representations (ICLR)</i> .	681
631	Direct alignment of draft model for speculative de-	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying	682
632	coding with chat-fine-tuned LLMs. <i>arXiv preprint</i>	Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gon-	683
633	<i>arXiv:2403.00858</i> .	zalez, Hao Zhang, and Ion Stoica. 2023. Efficient	684
634	Raghavv Goel et al. 2025. VocabTrim: Vocabulary	memory management for large language model serv-	685
635	pruning for efficient speculative decoding in LLMs.	ing with PagedAttention. In <i>ACM Symposium on</i>	686
636	<i>arXiv preprint arXiv:2506.22694</i> .	<i>Operating Systems Principles (SOSP)</i> .	687
637	Aaron Grattafiori et al. 2024. The Llama 3 herd of	Nathan Lambert, Jacob Morrison, Valentina Pyatkin,	688
638	models. <i>arXiv preprint arXiv:2407.21783</i> .	Shengyi Huang, Hamish Ivison, Faeze Brahman,	689
639	Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee,	Lester James V. Miranda, Alisa Liu, Nouha Dziri,	690
640	and Di He. 2024. REST: Retrieval-based specula-	Shane Lyu, et al. 2024. Tulu 3: Pushing frontiers in	691
641	tive decoding. In <i>Proceedings of the Conference of</i>	open language model post-training. <i>arXiv preprint</i>	692
642	<i>the North American Chapter of the Association for</i>	<i>arXiv:2411.15124</i> .	693
643	<i>Computational Linguistics (NAACL)</i> .	Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang,	694
644	Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul	Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou,	695
645	Arora, Steven Basart, Eric Tang, Dawn Song, and Ja-	Ning Ding, Yun Luo, and Yu Cheng. 2026. Draft-	696
646	cob Steinhart. 2021. Measuring mathematical prob-	OPD: On-policy distillation for speculative draft mod-	697
647	lem solving with the MATH dataset. In <i>Advances in</i>	els. <i>arXiv preprint arXiv:2605.29343</i> .	698
648	<i>Neural Information Processing Systems (NeurIPS),</i>	Yaniv Leviathan, Matan Kalman, and Yossi Matias.	699
649	<i>Datasets and Benchmarks Track</i> .	2023. Fast inference from transformers via spec-	700
650	Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015.	ulative decoding. In <i>International Conference on</i>	701
651	Distilling the knowledge in a neural network. <i>arXiv</i>	<i>Machine Learning (ICML)</i> .	702
652	<i>preprint arXiv:1503.02531</i> .	Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang,	703
653	Fenglu Hong, Ravi Raju, Jonathan Lingjie Li, Bo Li,	Fan Yin, Shuai Shi, et al. 2026a. SpecForge: A	704
654	Urmish Thakker, Avinash Ravichandran, Swayamb-	flexible and efficient open-source training frame-	705
655	hoo Jain, and Changran Hu. 2025. Training domain	work for speculative decoding. <i>arXiv preprint</i>	706
656	draft models for speculative decoding: Best practices	<i>arXiv:2603.18567</i> .	707
657	and insights. <i>arXiv preprint arXiv:2503.07807</i> .	Yuchen Li, Rui Kong, Zhonghao Lyu, Qiyang Li, Xin-	708
658	Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel	ran Chen, Hengyi Cai, Lingyong Yan, Shuaiqiang	709
659	Amar, and Yedid Hoshen. 2025. Charting and navi-	Wang, Jiashu Zhao, Guangxu Zhu, Linghe Kong,	710
660	gating Hugging Face’s model atlas. <i>arXiv preprint</i>	Guihai Chen, Haoyi Xiong, and Dawei Yin. 2026b.	711
661	<i>arXiv:2503.10633</i> .	FlexSpec: Frozen drafts meet evolving targets in	712
662	Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan	edge-cloud collaborative LLM speculative decoding.	713
663	Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and	<i>arXiv preprint arXiv:2601.00644</i> .	714
664	Weizhu Chen. 2022. LoRA: Low-rank adaptation of	Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang	715
665	large language models. In <i>International Conference</i>	Zhang. 2024a. EAGLE-2: Faster inference of lan-	716
666	<i>on Learning Representations (ICLR)</i> .	guage models with dynamic draft trees. In <i>Proceed-</i>	717
		<i>ings of the Conference on Empirical Methods in Nat-</i>	718
		<i>ural Language Processing (EMNLP)</i> .	719
		Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang	720
		Zhang. 2024b. EAGLE: Speculative sampling re-	721
		quires rethinking feature uncertainty. In <i>Interna-</i>	722
		<i>tional Conference on Machine Learning (ICML)</i> .	723

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. <i>arXiv preprint arXiv:2503.01840</i> .	780
Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In <i>International Conference on Learning Representations (ICLR)</i> .	782
Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. 2026. Efficiently aligning draft models via parameter- and data-efficient adaptation. <i>arXiv preprint arXiv:2603.09527</i> .	783
Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. 2024. Online speculative decoding. In <i>International Conference on Machine Learning (ICML)</i> .	784
Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, et al. 2024. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In <i>ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)</i> .	785
Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2024. SuffixDecoding: A model-free approach to speeding up large language model inference. <i>arXiv preprint arXiv:2411.04975</i> .	786
Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. 2025. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In <i>Advances in Neural Information Processing Systems (NeurIPS)</i> .	787
Red Hat AI and vLLM Project. 2025. Speculators: A unified library for speculative decoding algorithms in LLM serving. https://github.com/vllm-project/speculators .	788
Apoorv Saxena. 2023. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding .	789
Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. 2026. RL’s razor: Why online reinforcement learning forgets less. In <i>International Conference on Learning Representations (ICLR)</i> .	790
Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca .	791
Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhi-fang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In <i>Findings of the Association for Computational Linguistics: ACL 2024</i> .	792
An Yang et al. 2025. Qwen3 technical report. <i>arXiv preprint arXiv:2505.09388</i> .	793
Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024a. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In <i>Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)</i> .	794
Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. 2024b. Learning harmonized representations for speculative sampling. <i>arXiv preprint arXiv:2408.15766</i> .	795
Shuyu Zhang, Lingfeng Pan, Qicheng Wang, Yaqi Shi, Yueyang Tan, Ruyu Yan, Jiaqi Chen, Lixing Du, and Lu Wang. 2026. EvoSpec: Evolving speculative decoding via real-time vocabulary and parameter adaptation. <i>arXiv preprint arXiv:2605.27390</i> .	796
Weilin Zhao et al. 2025. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. <i>arXiv preprint arXiv:2502.14856</i> .	797
Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. DistillSpec: Improving speculative decoding via knowledge distillation. In <i>International Conference on Learning Representations (ICLR)</i> .	798
A Implementation Details	
Repair. We optimize Eq. (2) with AdamW (learning rate 2×10^{-5} , weight decay 0.01, cosine schedule with 3% warm-up) for one epoch over the 16k repair examples, packing roughly 2k response tokens per step, with $k_{\max}=3$ training-time-test steps. The loss is computed on response positions only. Repair data are generated greedily with vLLM, capped at 512 response tokens, and deduplicated exactly and approximately against all evaluation prompts, including the full MATH-500 set. ReFit-interface updates $W_{fc} \in \mathbb{R}^{4096 \times 12288}$ (50.3M parameters); ReFit-full updates the interface, the decoder layer and the language-modeling head (424.8M parameters), keeping the token embedding shared with the target. When a released drafter omits the embedding table, the trainer loads the target’s embedding, exactly as the serving engine does. The three R1-Distill seeds use the same data and differ only in the training seed.	800
Component analysis. For Figure 2, all variants use the same 256 R1-Distill responses and 300 optimization steps, starting from the production EAGLE-3 drafter. LoRA adapters use rank 16 on	801

the query, value and MLP projections of the decoder layer. The feature-statistics baseline fits a per-layer affine map that matches the mean and scale of the derivative’s tapped features to the parent’s.

Crossover. For each model, we take 64 contexts generated by the derivative, run both the parent and the derivative on the identical text, and feed each model’s tapped hidden states to the frozen family drafter. Agreement is the rate at which the drafter’s first prediction matches a given model’s greedy next token. The feature-source effect averages the change in agreement from swapping parent features for derivative features under each fixed verifier; the policy effect averages the change from swapping the verifier under each fixed feature source.

Evaluation. All acceptance measurements use vLLM with greedy decoding, $K=4$ draft tokens (DFlash: its native block of ten), a maximum of 512 new tokens and batch size 8. Prompts that produce no speculative step are excluded pairwise across compared methods. Speedups are measured on 32 SPEED-Bench prompts at batch size 1 and 128 at batch size 8, each in three fresh server processes with three warm repetitions, relative to autoregressive decoding of the same prompts. The dedicated drafter’s training cost is estimated from our measured A40 training throughput for the public EAGLE-3 recipe at 10 to 40 epochs.

B Census Details

The census covers 174 derivatives in the Llama-3.1-8B and Qwen3-8B families that load with the architecture and tokenizer of the instruction-tuned parent. Post-training histories are taken from model cards and grouped as in Figure 1a. Retention is computed per derivative on 64 prompts from its own domain and on 128 SPEED-Bench prompts, and aggregated within each group. The two reasoning distills are evaluated on SPEED-Bench with 128 prompts. The compatibility probe ranks derivatives by acceptance retention on 16 fixed requests; we report its AUROC for identifying derivatives whose retention on the remaining, disjoint requests falls below 90%.